Exploring the Trade-offs Between Audio Fidelity and Computational Efficiency in a Custom Real-Time Browser-Based Synthesizer

Tim Fan (SID: 550790019)
https://github.com/fanchenhao2206/elec5305-project-550790019

How do different oscillator anti-aliasing techniques and digital filter architectures influence the relationship between audio fidelity and computational efficiency in real-time browser-based synthesis?

I. Literature Review

Digital signal processing (DSP) refers to the manipulation of signals represented in discrete form, such as those stored or processed by computers. In computer music, DSP provides the theoretical and practical foundation for digitally modelling and recreating the behaviour of analog audio systems, including oscillators and filters. However, accurately translating analog oscillators and filters into the digital domain presents several technical challenges. Specifically, the choice of oscillator anti-aliasing techniques and digital filter architectures introduces different considerations that influence the accuracy, stability, and computational efficiency of the resulting audio. These considerations become particularly important in real-time browser-based synthesis, where audio fidelity must be balanced against computational constraints.

This section examines these challenges through a review of existing literature, focusing on the methods or approaches used or proposed to address them.

A. Digital Oscillators

Analog oscillators generate the fundamental voltage signals used in analog synthesisers, while digital oscillators aim to reproduce equivalent waveforms in the discrete-time domain. Common waveforms include the sine, sawtooth, square and triangle waves, each with distinct harmonic characteristics that contribute to its perceived timbre.

A naive digital oscillator generates waveforms directly from mathematical functions or phase-based calculations. A sawtooth, for example, can be produced with a phase accumulator, which advances its phase on every sample by an amount determined by the oscillator frequency and the sample rate. The phase is mapped to an amplitude range and, once it reaches its maximum, wraps back to the minimum, producing the characteristic sawtooth shape. This abrupt jump is known as a *discontinuity*, and it introduces infinitely many high-frequency harmonics. The Nyquist–Shannon sampling theorem states that a signal can only be represented accurately if it contains no components above the Nyquist frequency, which is half the sampling frequency. A sawtooth always contains harmonics above this limit, so they cannot be represented and instead fold back into the audible range as inharmonic distortion known as aliasing.

The problem worsens as the fundamental frequency rises. This applies to any waveform with discontinuities, such as the sawtooth and also the square wave, because a greater proportion of their harmonics then lies above the Nyquist frequency.

B. Anti-Aliasing Techniques

Anti-aliasing refers to techniques that reduce the effect of aliasing. One approach is the Band-Limited Step (BLEP) method. The idea is to replace the abrupt discontinuity in the naive waveform with a band-limited approximation, which reduces the high-frequency content introduced by the discontinuity and so improves the accuracy of the eventual reconstruction.

An ideal step is the integral of an impulse, and an ideal band-limited step is therefore the integral of a band-limited impulse, typically a windowed sinc function[1]. Rather than synthesising the entire waveform from band-limited components, the BLEP method computes only the *residual*—the difference between the band-limited step and the naive (non-band-limited) step—and applies it as a correction to the naive waveform in the immediate vicinity of each discontinuity, reducing the effect of aliasing.

A widely used variant is the polynomial BLEP (PolyBLEP), which approximates the residual with a low-order polynomial instead of a table of sinc-derived values[2]. For a normalised phase $t \in [0, 1)$ and a per-sample phase increment $\Delta t = f_0/f_s$, where f_0 is the oscillator frequency and f_s the sampling frequency, a two-sample PolyBLEP residual is given by

$$r(t) = \begin{cases} 2\dfrac{t}{\Delta t} - \left(\dfrac{t}{\Delta t}\right)^2 - 1, & 0 \le t < \Delta t, \\ \left(\dfrac{t-1}{\Delta t}\right)^2 + 2\dfrac{t-1}{\Delta t} + 1, & 1 - \Delta t < t < 1, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

The corrected sawtooth is then obtained by subtracting this residual from the ideal but naive sawtooth waveform $s(t)$

$$y(t) = s(t) - r(t) = (2t - 1) - r(t), \tag{2}$$

which also smoothes the wrap-around over the samples adjacent to the discontinuity. The naive sawtooth wave is chosen to

be $s(t) = 2t - 1$ for simplicity of residual calculation and thus implementation of PolyBLEP, because it reaches its minimal extreme of $s(t) = -1$ at $t = 0$, and the maximum extreme of $s(t) = 1$ at $t = 1$; the discontinuities exist at integer intervals.

A square wave may be treated as the sum of two sawtooth-like discontinuities, one rising and one falling, so the same residual is applied at both transitions (with opposite sign). A triangle wave, being continuous, has no step discontinuities, but its first derivative is discontinuous; this slope discontinuity can be corrected with an analogous polynomial band-limited ramp (PolyBLAMP) residual[3]

The principal advantages of PolyBLEP and PolyBLAMP are their low computational cost, the lack of lookup tables, and suitability for real-time use with modulated frequency, since the correction depends only on the current phase and phase increment. Its limitation is that a low-order polynomial only approximates the ideal band-limited step, so some residual aliasing remains, particularly at high fundamental frequencies, where it is attenuated rather than eliminated.

Alternative approaches offer different trade-offs. Oversampling reduces aliasing by generating waveforms at higher sampling rates before filtering and decimation, but increases computational cost [4]. Wavetable synthesis uses precomputed band-limited waveforms to suppress harmonics above the Nyquist frequency, but requires additional memory and table selection, reducing flexibility especially during dynamic modulation[5].

C. Digital Filter Design Architectures—Biquad Filter

Filters shape the harmonic content of an oscillator's output. A lowpass filter attenuates components above a cutoff frequency, a highpass filter attenuates those below it, and a bandpass filter passes a band around the cutoff while attenuating components on either side. In a synthesizer, the cutoff frequency and resonance are typically modulated in real time, so a digital filter must remain stable and accurate while its coefficients change from sample to sample.

The biquad is the most widely used digital filter structure. It is a second-order infinite impulse response (IIR) filter with a transfer function

$$H(z) = \frac{b_0 + b_1 z^{-1} + b_2 z^{-2}}{1 + a_1 z^{-1} + a_2 z^{-2}}, \tag{3}$$

whose coefficients are computed from the cutoff frequency, resonance and filter type, commonly using the closed-form expressions of the Audio EQ Cookbook[1]. Its advantages include low computational cost and the flexibility to obtain different filter responses through changes to the numerator coefficients.

However, its conventional difference-equation implementation presents some challenges under rapid real-time parameter modulation. Changes to the cutoff frequency or resonance require recalculation of the filter coefficients, while the internal delay states retain values generated under the previous coefficient configuration. This mismatch can introduce transient artefacts during modulation[6].

[1]https://www.w3.org/TR/audio-eq-cookbook/

D. Digital Filter Design Architectures—State Variable Filter

To address some of these limitations, the State Variable Filter (SVF) can be implemented using a zero-delay feedback (ZDF) topology and a topology-preserving transform (TPT). Rather than implementing the filter as a conventional difference equation by remembering past outputs, this structure instead remembers integrator-equivalent states to calculate the lowpass, bandpass and highpass responses. Specifically, the trapezoidal integration approximation method allows the feedback structure to be discretised without introducing an additional unit-sample delay into the feedback path[7]. Taken from the same source[7], the analog SVF has the transfer functions

$$H_{\mathrm{LP}}(s) = \frac{1}{s^2 + 2Rs + 1} \tag{4}$$

$$H_{\mathrm{BP}}(s) = \frac{s}{s^2 + 2Rs + 1} \tag{5}$$

$$H_{\mathrm{HP}}(s) = \frac{s^2}{s^2 + 2Rs + 1} \tag{6}$$

where s is normalised to the cutoff frequency and $R = 1/(2Q)$ is the damping factor. Applying the trapezoidal rule to each integrator, with the cutoff frequency pre-warped to compensate for the nonlinear frequency mapping introduced by the discretisation, gives

$$g := \tan\left(\frac{\pi f_c}{f_s}\right) \tag{7}$$

so that the feedback loop can be solved for the current input sample $x[n]$ without a unit delay applied onto the outputs. Specificially, the highpass output is obtained in closed form

$$y_{\mathrm{HP}}[n] = \frac{x[n] - (2R + g)\, s_1[n-1] - s_2[n-1]}{1 + 2Rg + g^2} \tag{8}$$

where notice that each new output does not rely on the past outputs of the filter, and s_1 and s_2 denote the states of the two integrators. The bandpass and lowpass outputs follow from the two integrators in sequence

$$y_{\mathrm{BP}}[n] = g\, y_{\mathrm{HP}}[n] + s_1[n-1] \tag{9}$$

$$y_{\mathrm{LP}}[n] = g\, y_{\mathrm{BP}}[n] + s_2[n-1] \tag{10}$$

Finally, the integrator states are updated for the next sample by the following closed forms

$$s_1[n] = y_{\mathrm{BP}}[n] + g\, y_{\mathrm{HP}}[n] \tag{11}$$

$$s_2[n] = y_{\mathrm{LP}}[n] + g\, y_{\mathrm{BP}}[n] \tag{12}$$

so that $y[n+1]$ will be calculated from these new state values. To be clear then, the ZDF SVF remains an IIR filter—in the sense that state from previous samples are preserved for the calculation of new samples—but its topology offers advantages over conventional biquad difference equation implementations when filter parameters are modulated rapidly, improving filter stability.

II. Methods

This section describes the full approach used to answer the research question: how audio fidelity trades off against computational efficiency in a real-time, browser-based synthesizer. The approach has two parts. First, a synthesizer, *picoDAW*[2], was implemented so that every algorithm under study (naive, PolyBLEP and PolyBLAMP oscillators; Biquad IIR and ZDF state variable filters) runs inside the same signal path, on the same platform, under the same real-time constraints. Second, three controlled experiments were run on that synthesizer, measuring CPU cost, aliasing and filter stability. A final, cross-cutting question is how well these algorithms hold up within the Web Audio API and AudioWorklet environment.

Building a single shared platform, rather than benchmarking isolated reference code, is deliberate. Computational cost in a browser depends on the WebAssembly (WASM) compilation, the AudioWorklet scheduling model and the 128-sample block size, so a measurement taken outside this environment would not answer the question as posed.

A. Implementation of a Browser-Based Synthesizer

1) Platform and development process: picoDAW was implemented with the iPlug2[3] framework, compiled to WebAssembly with Emscripten. The project was built on top of the example instrument provided with iPlug2 (`IPlugInstrument`). The original goal was a multi-voice, multi-instrument synthesizer, but this proved too difficult within the project timeframe, and the scope was reduced to a single configurable voice architecture. No open-source reference implementation was used during development; a few comparable projects were found only while writing this report. **cite related implementations**[4]

A significant part of the effort went into understanding how iPlug2's WASM build is structured, in particular how the user interface and the audio code communicate. Components were built in the following order: a MIDI sequencer, a gate sequencer (controlling legato versus retrigger behaviour), and then the DSP components compared in the experiments.

2) Two-thread architecture and parameter pipeline: The WASM build of picoDAW runs on two threads.

- **The main JavaScript thread** runs the UI. It executes the compiled `main()` of `IPlugWasmUI` (emitted by `emcc` as `picoDAW-ui.js`) and handles all graphics and user input.
- **The AudioWorklet thread** is spawned by the main thread before the UI loop starts. It repeatedly invokes the `process` function of the `AudioWorkletProcessor` defined in `picoDAW-processor`. Each invocation calls `processBlock`, which is bound to the C++ `IPlugWasmDSP::processBlock` in the DSP module `picoDAW-dsp.js`.

The two threads share no memory and communicate by message passing through the AudioWorklet's `MessagePort`. For example, when the user changes a parameter through the UI, like the gain knob of the synthesizer, `IPlugWasmUI::SendParameterValueFromUI` calls the JavaScript function `setParam(paramIdx, value)` of `IPlugWasmController`, which posts a message of type `'param'` to the worklet:

```
setParam(paramIdx, value) {
  this.workletNode?.port.postMessage({
    type: 'param', paramIdx, value });
}
```

On the other side—the DSP AudioWorklet thread—a `_onMessage` handler checks the message type received on its port, and for `param`, calls `onParam`. Emscripten binds this to `IPlugWasmDSP::OnParamMessage`, which calls `IPlugAPIBase::SetParameterValue` and then the user-level `picoDAW::OnParamChange`. Finally, the message is routed to the actual DSP object through `mDSP.SetParam`, changing the parameters of the underlying filters and oscillators in response, for example. Understanding this pipeline was a prerequisite for the later experiments.

3) DSP structure and voice design: The DSP code is organised in three main files.

a) `picoDAW_DSP.h`: treats the DSP as a black box. It owns a `MidiSynth` instance (`mSynth`) that can hold multiple voices, the smoothers for parameters that are not set per voice, and a low-frequency oscillator (LFO).

b) `picoDAW_Voice.h`: defines a voice, which holds one instance of every algorithm under comparison and selects among them at run time through index parameters:

- **Oscillator cores** (via the `IOscillator` interface): the iPlug2 sine core; custom naive sawtooth, triangle and square cores; and custom band-limited sawtooth, triangle and square cores (PolyBLEP for the discontinuous waveforms, PolyBLAMP for the triangle).
- **Filter cores** (via the `IFilter` interface): the iPlug2 ZDF state variable filter, and a custom Biquad IIR filter, both with three states set by `mFilterTypeIdx`; lowpass, highpass or bandpass. An IIR Moog ladder filter and ZDF ladder filter were also implemented, but are not discussed in this report.

A voice is thus configured by setting `mSynthCoreIdx`, `mAntiAliasing`, `mFilterCoreIdx`, `mIsZDF`, and the filter cutoff and resonance. Additionally, two ADSR envelopes were added to the signal chain for musical expression; one on the voice amplitude, and one on the filter cutoff. The custom oscillators and filters can be found implemented in `picoDAW_Oscillator.h` and `picoDAW_Filter.h` respectively.

All of these are set through the parameter pipeline above. Because every algorithm sits behind a common interface, the experiments can swap one algorithm for another without changing anything else in the signal path. This is what makes the comparisons controlled.

4) Per-sample signal path and modulation: In each sample, the voice computes the oscillator output at the pitch-derived frequency, multiplies it by the amplitude envelope and gain, and passes it through the selected filter. The filter cutoff is

[2] picoDAW
[3] iplug2
[4] hello

modulated by two sources: the filter ADSR envelope and the LFO output. Both modulate in the octave domain:

$$f_c(n) = f_{c,0} \cdot 2^{\,e(n) + d\,\ell(n)} \tag{13}$$

where $f_{c,0}$ is the base cutoff, $e(n)$ the filter envelope value, $\ell(n)$ the LFO value, and d the LFO depth (fixed at 2 octaves). Cutoff and resonance are updated every sample. This is the most demanding case for a filter, because its coefficients change at audio rate, and it is what the filter stability experiment later stresses.

B. Experiment 1: Computational Effiency

a) Aim: Compare the CPU cost of the naive, PolyBLEP and PolyBLAMP oscillators, and of the Biquad IIR and ZDF SVF filters, within the real-time audio path.

b) Method: The execution time of each 128-sample block was measured with Chrome's built-in tracing, recorded and inspected in the Perfetto trace viewer, rather than by timing code inside the AudioWorkletProcessor. Instrumenting the worklet manually was avoided because it adds code to the very path being measured and depends on browser timer resolution.

For each configuration, the synthesizer was set to play a single constant note (for example, a high-pitched sawtooth for the anti-aliasing comparison), and a trace of about 30 seconds was recorded in Chrome while the note sounded. In Perfetto, the whole recording was selected and the slices table was filtered to the `AudioWorkletProcessor::Process` slices, one of which is emitted for every 128-sample block. Perfetto reports, for each slice name, the number of calls, the total wall duration and the mean wall duration, and the mean wall duration per call was recorded as the cost of that configuration. At a sample rate of 48 kHz, a 30-second trace contains 11,250 blocks per configuration (the call count in the trace matches this figure), so means are taken over a large number of blocks.

Each `Process` slice contains child slices. Of these, "author script execution" covers the JavaScript and WebAssembly code of the worklet[5] (including the C++ `processBlock`) and is the part affected by the algorithm under test, whereas the remainder is browser overhead. To provide a more realistic view of the overall effect of different algorithms, the complete `Process` duration was considered the key metric.

c) Justification: Cost per block, compared against the block's real-time budget (128 samples $\approx$ 2.67 ms at 48 kHz), shows directly how much headroom an algorithm leaves and so whether it is practical in a browser. The expectation, from the algorithm structures, is that the Biquad IIR is cheapest, since it needs only a handful of multiplications per sample, and the ZDF filter costs more, since it solves an implicit feedback equation per sample. Furthermore, that using PolyBLEP or PolyBLAMP anti-aliasing costs more than not using anti-aliasing at all. The experiment tests whether that expectation holds in WASM and by how much.

[5] why?

C. Experiment 2: Audio Fidelity (Aliasing)

a) Aim: Quantify how much aliasing each oscillator algorithm produces and how well PolyBLEP and PolyBLAMP suppress it.

b) Method: A high note—A6, fundamental frequency of $f_0 = 1760$Hz—was played on the anti-aliased and naive sawtooth core, with the filter bypassed so that the spectrum reflects the oscillator alone. High fundamentals are used because as discussed in the literature review, few harmonics will fit below the Nyquist frequency, so the rest fold back audibly into the spectrum if not anti-aliased.

The output was captured and analysed with an FFT, using a Hann window and averaging over several frames. Specifically, the output samples of the audio processor were recorded directly in the AudioWorklet, before they reach the browser's audio output, and offered to the user as a 30-second mono WAV download; recording begins when the audio context is initialised. Because the capture is lossless and taken before any system-level processing, the spectra reflect the oscillator alone.

In addition to visual inspection, an alias measure—which we define as the energy of the aliasing present—is computed by using the FFT output. Specifcally, any values that fell within three FFT bin widths of an in-band harmonic were discarded, since they cannot be told apart from the wanted signal. The aliasing energy of a recording was then defined as the sum of the squared spectral magnitude in a window of ± 2 bins around each remaining predicted alias frequency:

$$E_{\text{alias}} = \sum_{j} \sum_{b=-2}^{2} P^2[i_j + b] \tag{14}$$

where i_j is the bin nearest the j-th predicted alias frequency and P is the normalised magnitude spectrum. The two cores are compared by the improvement $10 \log_{10}(E_{\text{alias,naive}}/E_{\text{alias,PolyBLEP}})$ in dB.

c) Justification: The FFT directly shows the unwanted foldback components, and the same note played through different cores isolates the effect of the algorithm. The expectation is that the naive waveforms show a high alias floor, and that PolyBLEP (sawtooth, square) and PolyBLAMP (triangle) reduce these lines substantially. Combined with Experiment 1, this gives the cost and the benefit of each method, so the trade-off in the title can be stated quantitatively.

D. Experiment 3: Filter Stability

a) Aim: Compare how the Biquad IIR and ZDF SVF filters behave under extreme modulation, where coefficient updates are most likely to cause instability.

b) Intended method: A controlled version of this experiment would sweep cutoff and resonance rapidly through automation and monitor the output for non-finite samples, unbounded growth, and discontinuities (clicks), recording these automatically in the processor, for each filter in lowpass, highpass and bandpass mode.

c) Method as carried out: Implementing the automated version was not completed within the project timeframe, so a simpler informal stress test was used. The LFO modulating the filter cutoff was raised to audio rate (up to 1000Hz), resonance was set to its maximum (100%), and the base cutoff was lowered, which is the harshest combination of modulation depth, rate and feedback available in the synthesizer. The output was recorded with OBS as part of a screen capture, and the compressed MP3 audio track was extracted and inspected in MATLAB.

d) Limitations: This test is qualitative. Not only is the recording passed through the system audio path and lossy MP3 compression, the Web Audio output is limited to the range $[-1, 1]$, so the recording shows only that the output saturated, and cannot show how far the internal filter state had grown. The capture also passed through the system audio path and lossy MP3 compression. The result therefore establishes that a filter became unstable under extreme settings, but does not measure by how much one filter is more stable than another.

E. New Tools and Techniques Learned

To carry out this work, the following had to be learned from scratch:

- the iPlug2 framework, including its WASM build, the UI/DSP split, and the `IParam` parameter pipeline;
- the Web Audio `AudioWorklet` model and its `MessagePort`-based communication;
- the Emscripten toolchain, including how C++ methods are bound to JavaScript;
- implementation of PolyBLEP/PolyBLAMP band-limiting and of zero-delay-feedback filter structures[2][3][7];
- in-browser performance measurement and spectral analysis techniques.

III. Results

hello world

IV. Discussion

hello world

V. Conclusion

Hello world

References

[1] E. Brandt, "Hard sync without aliasing," *Proceedings of the International Computer Music Conference, Havana, Cuba*, 2001.

[2] V. Välimäki, "Perceptually informed synthesis of bandlimited classical waveforms using integrated polynomial interpolation," *The Journal of the Acoustical Society of America*, vol. 131, pp. 974–986, 2001.

[3] F. Esqueda, "Aliasing reduction in soft-clipping algorithms," *European Signal Processing Conference, Nice, France*, 2015.

[4] J. Kahles, "Oversampling for nonlinear waveshaping: Choosing the right filters," *Journal of the Audio Engineering Society*, vol. 67, pp. 440–449, 2019.

[5] A. Franck, "Higher-order integrated wavetable synthesis," *Proceedings of the 15th International Conference on Digital Audio EFfects (DAFx-12), York, UK*, 2012.

[6] R. Rabenstein, "Minimization of transient signals in recursive time-varying digital filters," *Circuits, Systems and Signal Processing*, vol. 7, pp. 345–359, 1988.

[7] V. Zavalishin, *The Art of VA Filter Design (rev. 2.1.2)*, 2020.